

**HostelHub**

Web-to-Native Android APK Architecture Guide

Executive & Technical Explanation: Capacitor Native Hybrid Bridge vs PWA (Progressive Web App)

ARCHITECTURE TYPE

Standalone Native Android APK (.apk)

PWA USAGE STATUS

NO PWA — 100% Native Capacitor Bridge Used

FRONTEND WEB FRAMEWORK

Angular v19 Standalone Components

NATIVE BRIDGE ENGINE

Capacitor v8 (By Ionic Framework Team)

CLOUD API SERVER

Render Cloud Platform (Node.js / Express)

DATABASE HOST

Supabase Managed PostgreSQL (AWS Data Center)

1. Why PWA (Progressive Web App) Was ELIMINATED

A common misconception is that web applications converted to mobile phones use **PWA (Progressive Web Apps)**. For HostelHub, **PWA technology was explicitly rejected and eliminated** due to key technical limitations.

✗ Why PWA is INSUFFICIENT for Enterprise Hostel Management:

- **Browser Dependency:** PWAs are essentially browser shortcuts that rely on Chrome/Safari service workers. They do not generate a real standalone .apk installer file.
- **Restricted Camera Hardware Access:** PWAs cannot natively hook into deep camera hardware APIs for photo attachments during maintenance tickets.
- **Session Resets:** PWAs frequently clear local storage when Android OS runs low on memory, forcing users to re-login repeatedly.
- **No Native Back-Button Control:** PWAs cannot intercept phone hardware back-button presses to show clean exit modals.

✓ **The Solution: Capacitor Native Android Bridge:** HostelHub is compiled into a **true, standalone Native Android Package (.apk)** using **Capacitor v8**. It installs directly on Android devices, runs outside browser URL bars, and accesses native hardware APIs directly.

2. The 3-Step Web-to-Native APK Conversion Process

This simple 3-step workflow explains how the Angular web application was compiled into the final HostelHub-Latest.apk installer.

PHASE	PROCESS NAME	WHAT HAPPENS UNDER THE HOOD
Step 1	Angular Production Build	Angular CLI compiles TypeScript, HTML templates, and Vanilla CSS tokens into minified, high-performance JavaScript bundles inside dist/frontend-user.
Step 2	Capacitor Native Sync	Capacitor engine (<code>npx cap sync</code>) copies the web build assets into the Android native project folder at <code>android/app/src/main/assets/public</code> and registers native plugins.
Step 3	Gradle APK Compilation	Android Studio's Gradle build system (<code>gradlew assembleDebug</code>) compiles Java/Kotlin wrapper classes, hooks native APIs, and builds the final .apk binary file.

```
[ Angular Source Code ] --> (ng build) --> [ Web Bundle ]
                               |
                               (npx cap sync)
                               ▼
[ Android Studio Project ] --> (gradlew assembleDebug) --> [ HostelHub.apk ]
```

3. Native Hardware Integrations (Capacitor Plugins)

By utilizing Capacitor native plugins, HostelHub directly interacts with mobile hardware APIs:

- `@capacitor/camera`: Direct access to phone camera hardware for capturing maintenance complaint issue photos and staff work completion proofs.
- `@capacitor/preferences`: Access to Android native `SharedPreferences` to securely persist JWT login tokens, preventing `WebView` session resets.
- `@capacitor/app`: Hardware back-button event listener enforcing the 2-step home tab navigation and exit confirmation modal guard.

4. Data Security & Infrastructure Summary

Cloud Server Infrastructure:

- **Database Host: Supabase Managed PostgreSQL** hosted on secure **Amazon Web Services (AWS) Data Centers** with automated daily backups.
- **API Application Server: Render Cloud Platform** (<https://hostelhub-0cyi.onrender.com>) handling Node.js/Express REST routing and HTTPS SSL/TLS encryption.
- **Password Encryption:** Passwords hashed with **bcryptjs (10 salt rounds)** — zero plaintext storage.